SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: MLA0302 |

COURSE CODE: Reinforcement Learning

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

CONSTRAINT - BASED PROBLEM-SOLVING

SOLUTIONS

2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

Python Program

```
import random


rewards = [5, 8, 12]

epsilons = [0.1, 0.3, 0.5]

trials = 500


for epsilon in epsilons:
    counts = [0, 0, 0]
    total_reward = 0


    for _ in range(trials):
```

```
        if random.random() < epsilon:
            action = random.randint(0, 2)
        else:
            action = rewards.index(max(rewards))

        counts[action] += 1
        total_reward += rewards[action]

    print("\nEpsilon =", epsilon)
    print("Action Counts:", counts)
    print("Total Reward:", total_reward)
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT3-01.py ========

Epsilon = 0.1
Action Counts: [9, 21, 470]
Total Reward: 5853

Epsilon = 0.3
Action Counts: [46, 51, 403]
Total Reward: 5474

Epsilon = 0.5
Action Counts: [88, 86, 326]
Total Reward: 5040
```

Fig 1.1 - Console output of Experiment 1

3.Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

Python Program

```
states = ["Start", "Road", "Destination"]

actions = ["Move", "Rest"]



transition = {

    "Start": {"Move": "Road"},

    "Road": {"Move": "Destination", "Rest": "Road"},

    "Destination": {}

}



reward = {

    "Start": -2,

    "Road": -1,

    "Destination": 20

}



energy = 10
```

```
state = "Start"


print("Robot Navigation")


while state != "Destination" and energy > 0:
    action = "Move"
    state = transition[state][action]
    energy -= 2
    print("State:", state,
          "Reward:", reward[state],
          "Remaining Energy:", energy)


print("Goal Reached")
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT3-02.py ==
Robot Navigation
State: Road Reward: -1 Remaining Energy: 8
State: Destination Reward: 20 Remaining Energy: 6
Goal Reached
```

Fig 2.1 - Console output of Experiment 2

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

Python Program

```
battery = 30

deliveries = 0



while battery >= 5:

    deliveries += 1

    battery -= 5

    print("Delivery", deliveries,

          "Battery Left:", battery)



print("\nTotal Deliveries =", deliveries)
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT3-03.py ====
Delivery 1 Battery Left: 25
Delivery 2 Battery Left: 20
Delivery 3 Battery Left: 15
Delivery 4 Battery Left: 10
Delivery 5 Battery Left: 5
Delivery 6 Battery Left: 0

Total Deliveries = 6
```

Fig 3.1 - Console output of Experiment 3

8. Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

Python Program

```
import random


rewards = [5, 10, 15]


print("Random Strategy")

random_reward = 0


for i in range(20):

    action = random.randint(0,2)

    random_reward += rewards[action]


print("Total Reward =", random_reward)


print("\nEpsilon Greedy Strategy")


epsilon = 0.2
```

```
greedy_reward = 0


for i in range(20):
    if random.random() < epsilon:
        action = random.randint(0,2)
    else:
        action = rewards.index(max(rewards))

    greedy_reward += rewards[action]

print("Total Reward =", greedy_reward)
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT3-04.py ========
Random Strategy
Total Reward = 220

Epsilon Greedy Strategy
Total Reward = 260
```

Fig 4.1 - Console output of Experiment 4

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

Python Program

```
threshold = 100

usage = [20, 30, 15, 40, 25]



total = 0



for u in usage:

    total += u



    if total <= threshold:

        reward = 10

    else:

        reward = -20



    print("Usage:", total,

          "Reward:", reward)



print("\nFinal Consumption =", total)
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT3-05.py =======
Usage: 20 Reward: 10
Usage: 50 Reward: 10
Usage: 65 Reward: 10
Usage: 105 Reward: -20
Usage: 130 Reward: -20

Final Consumption = 130
```

Fig 5.1 - Console output of Experiment 5